

MICROSERVICES COURSE

Spring Boot & Spring Cloud — Professional Edition

Session 14

GitOps & Deployments

Dr. ElSayed Mohamed Elsayed Baladoh

Tech Lead · Ph.D. · sayedbaladoh@yahoo.com

Session 14 — GitOps & Deployments

Session 13 answered:

How do we automatically turn source code into a tested Docker image?

The result was:

```
Developer
  ↓
Git Push
  ↓
GitHub Actions
  ↓
Build + Test
  ↓
Docker Image
  ↓
Container Registry
```

But we stopped there.

Now we have a new problem:

How do we reliably deploy that image to Kubernetes?

Session 14 introduces **GitOps** as the answer.

1. The Problem Session 14 Solves

Suppose GitHub Actions creates:

```
ghcr.io/company/product-service:sha-a81c92d
```

The image is safely stored in GHCR.

But Kubernetes is still running:

```
product-service:sha-72a91bc
```

How does Kubernetes know that it should now run:

```
product-service:sha-a81c92d
```

One simple approach is to let CI execute:

```
kubectl set image ...
```

or:

```
kubectl apply -f deployment.yaml
```

That is a **push-based deployment** model.

The architecture would be:

```
GitHub Actions
  |
  | kubectl
  v
Kubernetes
```

This works, but it creates an important coupling:

```
CI
|
+---- knows about Kubernetes
|
+---- needs Kubernetes credentials
|
+---- changes the cluster directly
```

GitOps takes a different approach.

2. The GitOps Idea

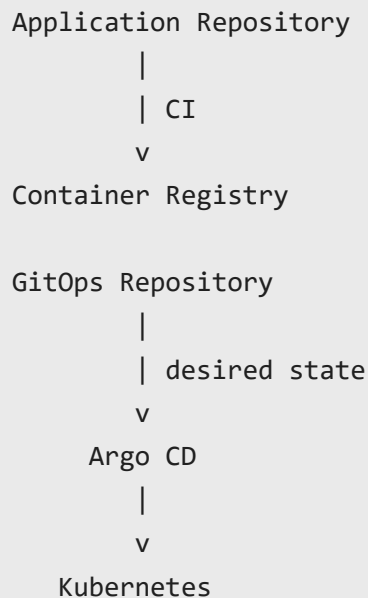
GitOps says:

Git contains the desired state of the system.

Instead of GitHub Actions directly telling Kubernetes what to do:

CI → Kubernetes

we introduce a Git repository containing the deployment configuration:



The key idea is:

Git = Desired State

3. What Does "Desired State" Mean?

This is one of the most important Kubernetes concepts.

Suppose we want:

```
product-service
replicas = 3
image = sha-a81c92d
```

```
port = 8080
```

That is the state we **want**.

We can represent it in a Kubernetes Deployment:

```
spec:
  replicas: 3
  template:
    spec:
      containers:
        - name: product-service
          image: ghcr.io/company/product-service:sha-a81c92d
```

This says:

Kubernetes, I want three Product Service Pods running this image.

That is the **desired state**.

4. Desired State vs Actual State

Kubernetes has:

Desired State

What the manifests say should exist.

```
3 Product Service replicas
```

Actual State

What is really running in the cluster.

For example:

```
Pod 1 → Running
Pod 2 → Running
```

Pod 3 → CrashLoopBackOff

So:

Desired:
3 healthy replicas

Actual:
2 healthy replicas

Kubernetes controllers continuously work to move:

Actual State
↓
toward
↓
Desired State

This idea of continuously trying to reach a declared state is central to Kubernetes.

5. Where GitOps Adds Value

With GitOps, the desired state is stored in Git.

For example:

GitOps Repository

product-service/
└─ deployment.yaml

The file says:

```
replicas: 3  
image: ghcr.io/company/product-service:sha-a81c92d
```

Git therefore becomes a historical record of what we intended to deploy.

We can see:

```
Commit A
  ↓
product-service:sha-111111

Commit B
  ↓
product-service:sha-222222

Commit C
  ↓
product-service:sha-333333
```

This gives us a deployment history.

6. The Two Git Repositories

For our course architecture, it is useful to distinguish two repositories.

Application Repository

Contains source code:

```
microservices-pro-platform
```

For example:

```
services/
├─ product-service/
├─ order-service/
└─ payment-service/
```

Its responsibility is:

```
Source Code
Build
```

Tests
Dockerfile

GitOps Repository

Contains deployment configuration.

For example:

microservices-pro-gitops

It might contain:

```
k8s/  
├─ product-service/  
│   ├── deployment.yaml  
│   ├── service.yaml  
│   └─ kustomization.yaml  
│  
├─ order-service/  
│   ├── deployment.yaml  
│   └─ service.yaml
```

Its responsibility is:

Deployment Configuration
Desired State
Environment Configuration

7. Why Separate the Repositories?

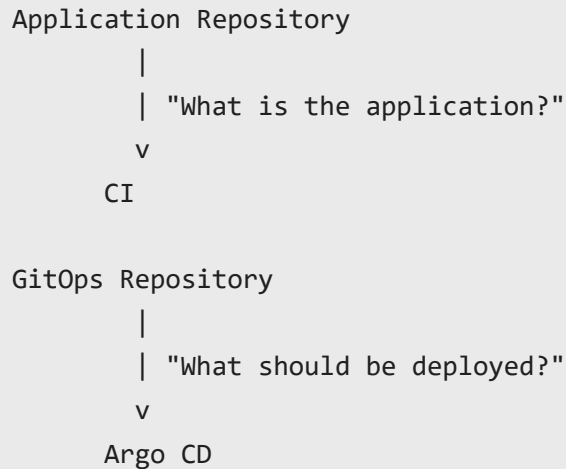
Imagine we put everything in one repository:

source code
Dockerfile

Kubernetes manifests
production configuration

This is possible.

But separating them gives us a cleaner responsibility model:



This separation is particularly useful when:

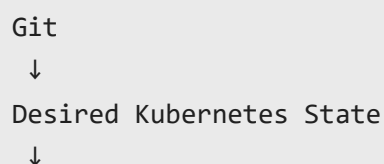
- different teams own application code and infrastructure;
- production deployment needs additional approval;
- infrastructure changes need a separate audit trail;
- multiple environments need different configurations.

8. What Is GitOps?

A simple student definition:

GitOps is a deployment approach where Git stores the desired state of the system, and an automated agent continuously makes the running environment match that desired state.

In our course:



Argo CD

↓

Kubernetes

9. What Is Argo CD?

Now we introduce the tool that implements GitOps for our Kubernetes environment.

Argo CD is a Kubernetes-focused continuous delivery tool.

Its job is essentially:

Watch Git

↓

Read desired state

↓

Compare with Kubernetes

↓

Synchronize when necessary

Think of Argo CD as a **GitOps controller**.

10. Argo CD's Mental Model

Do not think:

"Argo CD is another CI server."

Instead think:

GitHub Actions

=

Build & Test

Argo CD

=

Deploy & Reconcile

So:

GitHub Actions

|

| produces image

v

Container Registry

Argo CD

|

| reads desired state

v

Kubernetes

11. The Complete Flow

Now combine Sessions 13 and 14.

APPLICATION SIDE

Developer

|

| git push

v

Application Repository

|

v

GitHub Actions

|

+--> Compile

|

+--> Test

|

+--> Quality Gate

|

```
+--> Docker Build
|
+--> Docker Push
|
v
Container Registry
```

Then:

```

GITOPS SIDE

GitOps Repository
|
| desired state
v
Argo CD
|
| reconcile
v
Kubernetes
|
v
Pods
```

There is an important missing connection:

Something has to update the GitOps repository with the new image version.

For example:

```
product-service:sha-OLD
```

becomes:

```
product-service:sha-NEW
```

That update can be automated.

12. Image vs Deployment Configuration

This distinction is critical.

The Docker registry contains:

```
product-service:sha-a81c92d
```

The GitOps repository contains something like:

```
image:  
  repository: ghcr.io/company/product-service  
  tag: sha-a81c92d
```

So:

```
Registry  
  |  
  | stores  
  v  
Docker Image  
  
GitOps Repository  
  |  
  | declares  
  v  
Which image Kubernetes SHOULD run
```

The registry stores the artifact.

Git stores the desired deployment state.

13. What Happens During a New Release?

Suppose the current production image is:

```
sha-111111
```

A developer creates a new version.

CI produces:

```
sha-222222
```

Now we update GitOps:

```
- image: ghcr.io/company/product-service:sha-111111  
+ image: ghcr.io/company/product-service:sha-222222
```

That commit changes the desired state.

Argo CD sees:

```
Git says:  
sha-222222
```

```
Cluster says:  
sha-111111
```

There is a difference.

Argo CD can synchronize the cluster.

14. What Is Reconciliation?

Reconciliation means repeatedly comparing:

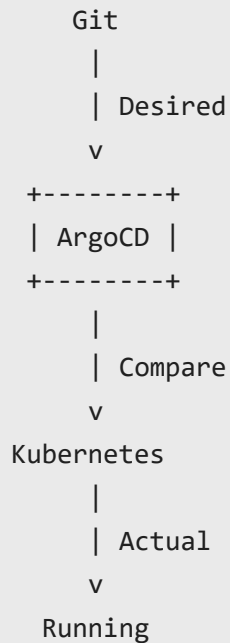
```
Desired State
```

with:

```
Actual State
```

and taking action when they differ.

Conceptually:



If they match:

`Desired = Actual`

then the system is synchronized.

If they differ:

`Desired ≠ Actual`

Argo CD can take action to bring the cluster back toward the desired state.

15. What Is Drift?

Suppose Git says:

`replicas: 3`

But someone manually runs:

```
kubectl scale deployment product-service --replicas=5
```

Now:

```
Git:  
3 replicas
```

```
Cluster:  
5 replicas
```

The cluster has **drifted** from the desired state.

GitOps gives us a mechanism to detect this difference.

This is one of the major benefits of GitOps.

16. Why Is Drift Detection Important?

Without GitOps, someone may manually change production:

```
kubectl edit deployment product-service
```

Six months later:

"Why is production configured this way?"

Nobody knows.

With GitOps, the intended configuration is stored in Git.

We can ask:

```
Who changed the deployment?  
When?  
What changed?  
Why?  
Which commit introduced it?
```

This improves:

- auditing;
- reproducibility;
- troubleshooting;
- collaboration.

17. The Golden Rule

A useful rule for students:

Do not manually modify Kubernetes resources that are managed by GitOps.

Instead:

```
Need to change replicas?  
  ↓  
Change Git  
  ↓  
Commit  
  ↓  
Argo CD  
  ↓  
Kubernetes
```

Not:

```
Need to change replicas?  
  ↓  
kubectl edit
```

The second approach creates drift.

18. What Is a Kubernetes Manifest?

Before creating an Argo CD application, students need to understand the files Argo CD will deploy.

A **Kubernetes manifest** is a YAML document describing a Kubernetes resource.

For example:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: product-service
spec:
  replicas: 3
```

This describes a Kubernetes Deployment.

Another file could describe a Service:

```
apiVersion: v1
kind: Service
metadata:
  name: product-service
```

So a manifest is essentially:

A declarative description of a Kubernetes resource.

19. Declarative vs Imperative

This distinction is extremely important.

Imperative

Tell the system exactly what command to execute.

For example:

```
kubectl scale deployment product-service --replicas=3
```

You are saying:

Execute this operation.

Declarative

Describe the state you want:

```
spec:  
  replicas: 3
```

You are saying:

I want three replicas.

Kubernetes determines what actions are necessary to achieve that state.

GitOps strongly favors the declarative model.

20. Why Declarative Configuration Fits Git

A declarative file can be committed:

```
git add deployment.yaml  
git commit -m "Scale product service to three replicas"  
git push
```

Now Git records:

```
Before:  
replicas = 2
```

```
After:  
replicas = 3
```

That is much easier to audit than:

```
kubectl scale ...
```

executed manually on someone's laptop.

21. Basic Product Service Deployment

A simplified Deployment could look like:

```
apiVersion: apps/v1
kind: Deployment

metadata:
  name: product-service

spec:
  replicas: 3

  selector:
    matchLabels:
      app: product-service

  template:
    metadata:
      labels:
        app: product-service

    spec:
      containers:
        - name: product-service
          image: ghcr.io/company/product-service:sha-a81c92d
          ports:
            - containerPort: 8080
```

The important part for GitOps is:

```
image: ghcr.io/company/product-service:sha-a81c92d
```

That tells Kubernetes which image we want.

22. What Does Argo CD Actually Watch?

Argo CD can be configured to watch:

```
Git Repository
|
+-- Path
|
+-- Kubernetes manifests
```

For example:

```
GitOps Repository
└─ environments/
   └─ production/
      └─ product-service/
         ├── deployment.yaml
         └── service.yaml
```

Argo CD is told:

```
Repository:
microservices-pro-gitops

Path:
environments/production/product-service
```

It then reads the Kubernetes configuration from that location.

23. What Is an Argo CD Application?

Argo CD introduces an important concept called an **Application**.

An Argo CD Application tells Argo CD:

This Git repository/path represents the desired state for this Kubernetes application.

Conceptually:

```
Argo CD Application
```

```
|
+----- Git repository
|
+----- Git path
|
+----- Kubernetes cluster
|
+----- Namespace
```

For example:

Application:
product-service-production

Source:
GitOps repository

Path:
environments/production/product-service

Destination:
Kubernetes cluster

Namespace:
production

24. The Argo CD Application CRD

Argo CD itself extends Kubernetes with custom resources.

One of those resources is:

```
kind: Application
```

For example:

```
apiVersion: argoproj.io/v1alpha1
```

```
kind: Application
metadata:
  name: product-service
  namespace: argocd

spec:
  project: default

  source:
    repoURL: https://github.com/company/microservices-pro-gitops.git
    path: environments/production/product-service
    targetRevision: main

  destination:
    server: https://kubernetes.default.svc
    namespace: production
```

Don't memorize it yet.

Understand what it says.

25. Read the Application YAML Like English

This:

```
source:
  repoURL: ...
```

means:

Where is the desired configuration stored?

This:

```
path: environments/production/product-service
```

means:

Which directory contains the configuration?

This:

```
targetRevision: main
```

means:

Which Git revision should be used?

This:

```
destination:  
  server: ...
```

means:

Which Kubernetes cluster should receive the resources?

And:

```
namespace: production
```

means:

Which namespace should be used as the destination.

26. Sync

Now we reach another important Argo CD term:

Sync

Synchronization means:

```
Git desired state  
  ↓  
  compare  
  ↓  
Kubernetes actual state  
  ↓
```

apply necessary changes

For example:

Git:
image = sha-222222

Cluster:
image = sha-111111

After synchronization:

Cluster:
image = sha-222222

Now:

Desired State = Actual State

27. Manual Sync vs Automatic Sync

Argo CD can work in different modes.

Manual Sync

Argo CD detects:

OutOfSync

but waits for an operator to synchronize.

This gives:

Git change
↓
Argo CD detects change

↓
Human approves/syncs
↓
Kubernetes

Automatic Sync

Argo CD can automatically synchronize.

Then:

Git change
↓
Argo CD detects change
↓
Automatic synchronization
↓
Kubernetes

This is useful for environments where fully automated deployment is desired.

28. Why Start with Manual Sync?

For students, manual synchronization is useful initially because they can observe:

Git changed
↓
Argo CD detects
↓
Application becomes OutOfSync
↓
Sync
↓
Application becomes Synced

It makes the GitOps lifecycle visible.

Later, automatic synchronization can be enabled.

29. Synced vs OutOfSync

Argo CD commonly shows an application status.

Synced

Means the running cluster corresponds to the desired configuration Argo CD is tracking.

Conceptually:

```
Git = Cluster
```

OutOfSync

Means there is a difference.

For example:

```
Git:  
replicas = 3
```

```
Cluster:  
replicas = 2
```

Argo CD reports that the application is out of sync.

30. Healthy vs Unhealthy

There is another important distinction.

An application can be:

```
Synced
```

but:

Unhealthy

For example, Kubernetes may have applied the desired Deployment, but the Pods may be failing.

Therefore:

SYNC STATUS

and:

HEALTH STATUS

are not the same thing.

Think:

Synced

=

Configuration matches desired state.

Healthy

=

Resources are functioning as expected.

This distinction is extremely important when troubleshooting.

31. The Complete GitOps Lifecycle

Now put everything together.

1. Developer changes code

↓

2. GitHub Actions runs

↓

3. Tests pass

↓

4. Docker image is built

↓

```
5. Image pushed to GHCR
  ↓
6. GitOps manifest is updated
  ↓
7. Git commit is created
  ↓
8. Argo CD detects change
  ↓
9. Application becomes OutOfSync
  ↓
10. Argo CD synchronizes
  ↓
11. Kubernetes updates resources
  ↓
12. Pods start
  ↓
13. Argo CD observes health
  ↓
14. Application becomes Synced/Healthy
```

That is the heart of Session 14.

32. But How Does the GitOps Repository Get Updated?

This is an important practical question.

Suppose CI creates:

```
product-service:sha-222222
```

The GitOps repository still says:

```
product-service:sha-111111
```

Something needs to update the manifest.

A common approach is:

```
GitHub Actions
  |
  | update image tag
  v
GitOps Repository
```

For example, CI can modify:

```
image:
  tag: sha-222222
```

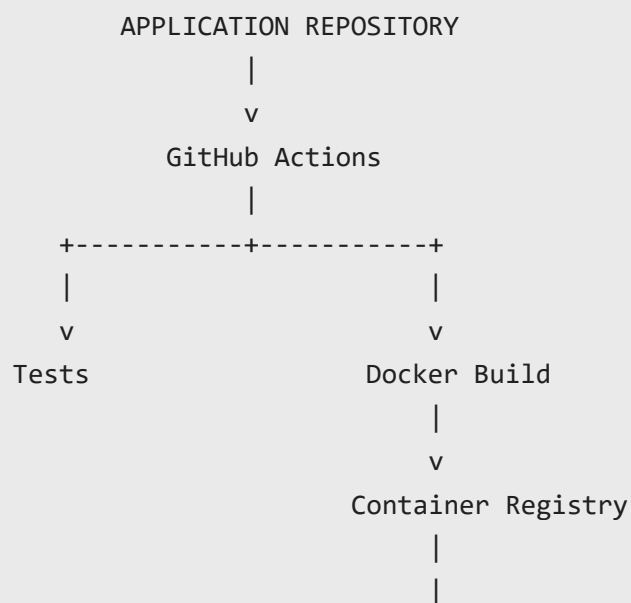
Then commit:

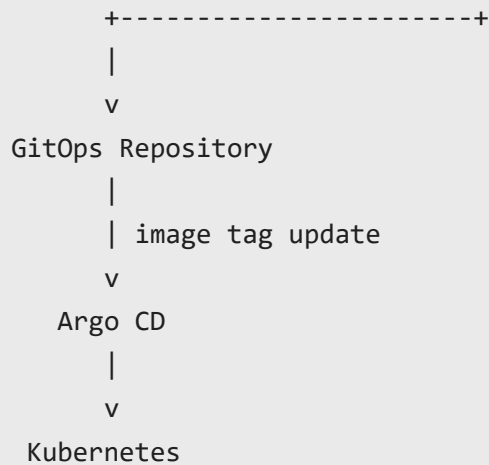
```
Update product-service image to sha-222222
```

Now Argo CD detects that Git change.

33. CI and GitOps Together

The complete architecture becomes:





The important point is:

CI produces the artifact. GitOps controls the deployment state.

34. Why Not Put the Image Tag Directly in CI and Kubernetes?

Because GitOps wants Git to remain the authoritative record of the desired deployment.

Suppose CI executes:

```
kubectl set image ...
```

Now the cluster changed.

But Git still says:

```
sha-111111
```

while the cluster runs:

```
sha-222222
```

Now Git no longer represents reality.

GitOps avoids this by making the deployment change itself a Git change.

35. What Is GitOps Drift?

Imagine:

```
Git:
product-service = sha-222222
```

```
Kubernetes:
product-service = sha-222222
```

Everything is synchronized.

Then an administrator manually changes Kubernetes:

```
Kubernetes:
product-service = sha-333333
```

Now:

```
Git:
sha-222222
```

```
Cluster:
sha-333333
```

Argo CD can detect:

```
OutOfSync
```

This is **drift**.

36. Reconciliation Can Correct Drift

If automated self-healing is configured, Argo CD can restore the desired state.

Conceptually:

```
Git
|
| desired = sha-222222
v
Argo CD
|
| sees drift
v
Kubernetes
|
| currently sha-333333
v
Reconcile
|
v
sha-222222
```

This is one of the strongest ideas behind GitOps.

37. Is Argo CD Replacing Kubernetes?

No.

Argo CD does not replace Kubernetes.

Think of the responsibilities:

```
Kubernetes
=
Runs and manages workloads.
```

while:

```
Argo CD
=
Synchronizes Kubernetes with Git.
```

And:

```
Git
=
Stores desired configuration.
```

So:

```
Git
↓
Argo CD
↓
Kubernetes
↓
Pods
```

38. Is Argo CD Replacing GitHub Actions?

No.

They have different responsibilities.

GitHub Actions

```
Build
Test
Package
Publish
```

Argo CD

```
Deploy
Synchronize
Detect drift
Reconcile
```

A professional pipeline can use both:

rather than trying to make one tool do everything.

39. Deployment Strategies

Once we have GitOps, we can begin discussing **how** to release new versions.

The simplest strategy is:

Rolling Update

Suppose:

Version 1
3 Pods

We release:

Version 2

Kubernetes gradually replaces the old Pods.

Conceptually:

```
v1 v1 v1
↓
v2 v1 v1
↓
v2 v2 v1
↓
v2 v2 v2
```

This is the standard Kubernetes rolling-update approach.

40. Why Rolling Updates Are Useful

Advantages:

- No need to stop all replicas at once.
- Simple.
- Native Kubernetes capability.
- Easy to understand.
- Suitable for many services.

But there is a limitation.

During the rollout, traffic may gradually move from:

v1

to:

v2

There is not necessarily fine-grained control over exactly what percentage of users receive v2.

That leads us to Canary deployments.

41. What Is Canary Deployment?

A **Canary deployment** means introducing a new version to a small portion of traffic or capacity first.

For example:

95% → v1

5% → v2

Then:

90% → v1

10% → v2

Then:

50% → v1

50% → v2

Eventually:

0% → v1

100% → v2

The idea is:

Expose the new version gradually and observe it before sending all traffic to it.

42. Important: Replica Canary vs Traffic Canary

This distinction is extremely important.

Suppose we have:

v1 = 9 replicas

v2 = 1 replica

We might say:

90/10

But that does **not necessarily mean exactly 90%/10% of user traffic**.

Why?

Because traffic distribution depends on:

- Service load balancing;
- connection behavior;
- request patterns;
- clients;
- networking layer.

So:

```
9 replicas v1  
1 replica v2
```

is a **capacity/replica approximation**, not necessarily a precise traffic split.

A true traffic canary requires traffic-management capabilities.

This distinction should be clear to students.

43. Kubernetes Service and Traffic

Suppose:

```
Service:  
product-service
```

selects:

```
app=product-service
```

and we have:

```
v1 Pods  
v1 Pods  
v1 Pods  
v2 Pod
```

The Service may distribute requests among matching Pods.

But Kubernetes Service itself is not a sophisticated percentage-based canary controller.

For precise traffic control, technologies such as:

```
Ingress controllers  
Service mesh  
Gateway/API routing  
Progressive delivery tools
```

may be used.

This becomes important later in the course.

44. Why Does Session 14 Introduce Canary?

Because after students understand:

GitOps
Argo CD
Deployment

the next engineering question is:

How can we release a new version safely?

Canary is one answer.

But we should not give students the false impression that:

replicas: 9

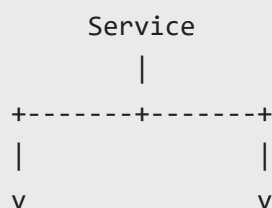
means exactly:

90% of requests

It is an approximation when using ordinary Kubernetes Service behavior.

45. Example Canary Architecture

Conceptually:



Version 1	Version 2
9 replicas	1 replica

This gives the new version a smaller amount of capacity.

We can then observe:

- Errors
- Latency
- CPU
- Memory
- Business metrics

before increasing the new version.

46. What Should We Monitor During Canary?

A canary is useful only if we observe the system.

Typical signals include:

Error rate

HTTP 5xx

Latency

p95
p99

Resource usage

CPU
Memory

Application metrics

For example:

```
orders failed
payments failed
product searches failed
```

Logs

```
exceptions
timeouts
connection failures
```

This connects Session 14 to the observability topics later in the course.

47. Example Release

Current version:

```
v1
```

New version:

```
v2
```

Initial rollout:

```
v1 = 9 replicas
v2 = 1 replica
```

Observe:

Error rate
Latency
Logs

If healthy:

v1 = 7
v2 = 3

Then:

v1 = 5
v2 = 5

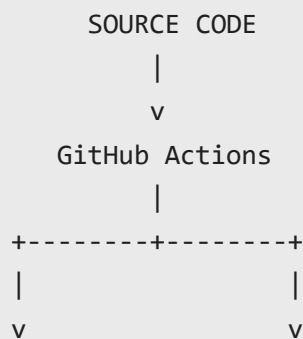
Eventually:

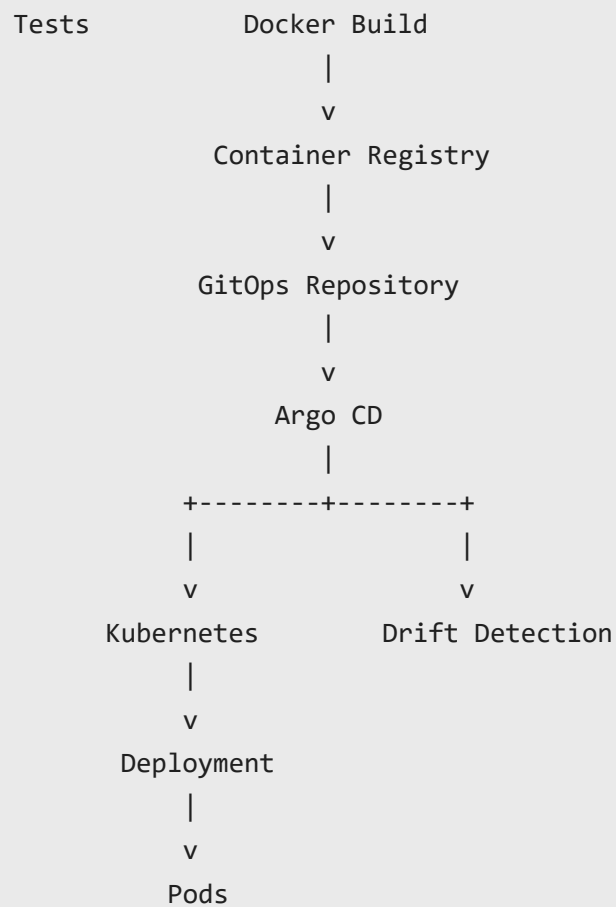
v1 = 0
v2 = 10

Again, this illustrates gradual capacity migration, not an exact percentage traffic controller.

48. What Students Should Understand by the End

At the end of Session 14, students should be able to explain this diagram without memorizing it:





And they should understand:

GitHub Actions
=
CI / artifact production
GitOps Repository
=
Desired deployment state
Argo CD
=
Reconciliation / deployment
Kubernetes
=
Runtime platform

49. Session 14 Practical Lab Flow

The student lab should follow this progression.

Lab 1 — Create the GitOps Repository

Create:

```
microservices-pro-gitops
```

with:

```
environments/  
└─ dev/  
    └─ product-service/  
        ├── deployment.yaml  
        └─ service.yaml
```

Lab 2 — Create the Deployment Manifest

Define:

```
Product Service  
3 replicas  
container port 8080
```

and the appropriate image.

Lab 3 — Install Argo CD

Deploy Argo CD into the Kubernetes cluster.

Conceptually:

```
Kubernetes
```

```
└─ argocd
   └─ Argo CD server
   └─ controllers
   └─ supporting components
```

Lab 4 — Create an Argo CD Application

Create:

```
product-service-dev
```

pointing to:

```
GitOps repository
+
dev/product-service
```

Lab 5 — Synchronize

Observe:

```
OutOfSync
```

Then synchronize.

Observe:

```
Synced
```

and:

```
Healthy
```

Lab 6 — Change Git

Change:

```
replicas: 3
```

to:

```
replicas: 4
```

Commit and push.

Observe Argo CD.

Expected:

```
Git changed
  ↓
Argo CD detects difference
  ↓
OutOfSync
  ↓
Sync
  ↓
4 replicas
```

Lab 7 — Create Drift

Manually change the Kubernetes deployment:

```
kubectl scale deployment product-service --replicas=5
```

Git still says:

```
replicas: 4
```

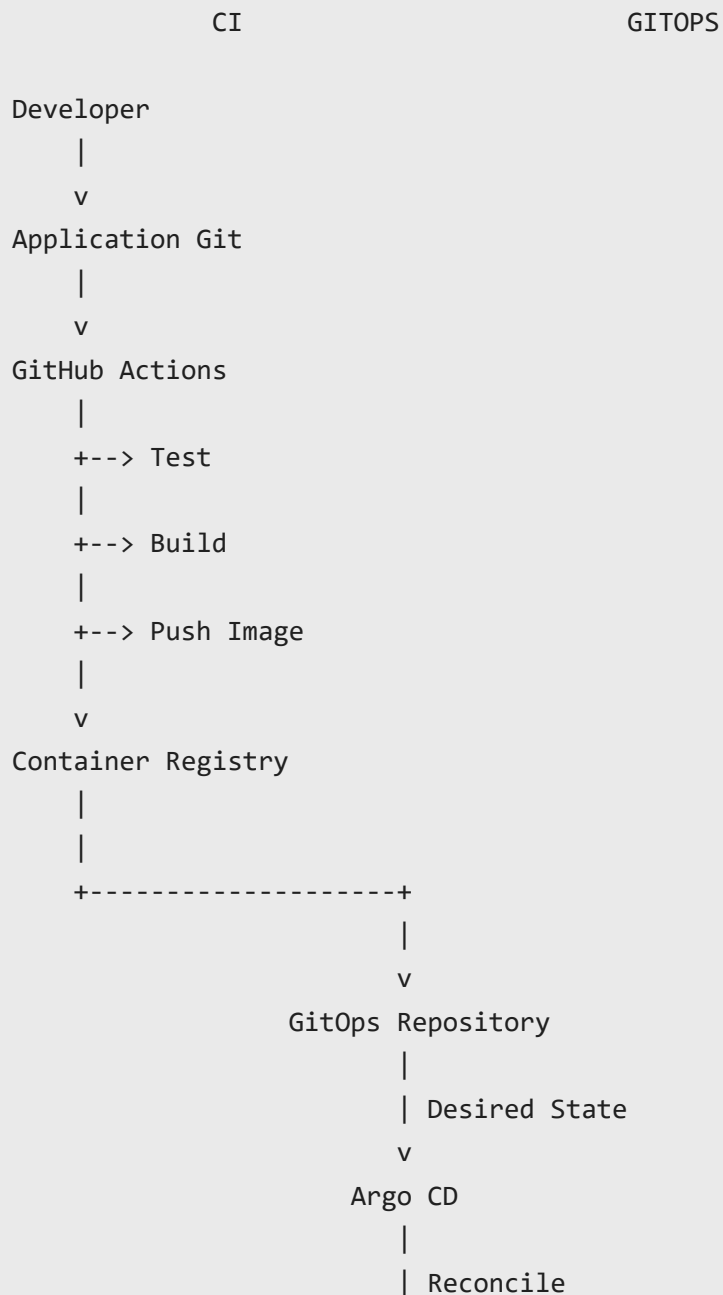
Observe the difference in Argo CD.

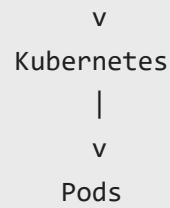
This demonstrates **drift**.

50. Session 14 — Final Mental Model

The most important thing to remember is not the Argo CD YAML.

It is this:





The core principle is:

CI produces the application artifact. GitOps declares what should run. Argo CD reconciles that declaration with Kubernetes.

And the next major question is:

How do we package these Kubernetes manifests so that managing many microservices and multiple environments does not become a huge YAML-maintenance problem?

That leads naturally to **Session 15 — Helm & Kubernetes Packaging**.